

code/ecf/sigsuspend.c

```
1  #include "csapp.h"
2
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)
6  {
7      int olderrno = errno;
8      pid = Waitpid(-1, NULL, 0);
9      errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
15
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         Sigprocmask(SIG_BLOCK, &mask, &prev); /* Block SIGCHLD */
27         if (Fork() == 0) /* Child */
28             exit(0);
29
30         /* Wait for SIGCHLD to be received */
31         pid = 0;
32         while (!pid)
33             sigsuspend(&prev);
34
35         /* Optionally unblock SIGCHLD */
36         Sigprocmask(SIG_SETMASK, &prev, NULL);
37
38         /* Do some work after receiving SIGCHLD */
39         printf(".");
40     }
41     exit(0);
42 }
```

code/ecf/sigsuspend.c

图 8-42 用 sigsuspend 来等待信号

8.6 非本地跳转

C 语言提供了一种用户级异常控制流形式，称为非本地跳转(nonlocal jump)，它将控制直接从一个函数转移到另一个当前正在执行的函数，而不需要经过正常的调用-返回序

列。非本地跳转是通过 `setjmp` 和 `longjmp` 函数来提供的。

```
#include <setjmp.h>

int setjmp(jmp_buf env);
int sigsetjmp(sigjmp_buf env, int savesigs);
```

返回: `setjmp` 返回 0, `longjmp` 返回非零。

`setjmp` 函数在 `env` 缓冲区中保存当前调用环境, 以供后面的 `longjmp` 使用, 并返回 0。调用环境包括程序计数器、栈指针和通用目的寄存器。出于某种超出本书描述范围的原因, `setjmp` 返回的值不能被赋值给变量:

```
rc = setjmp(env); /* Wrong! */
```

不过它可以安全地用在 `switch` 或条件语句的测试中[62]。

```
#include <setjmp.h>

void longjmp(jmp_buf env, int retval);
void siglongjmp(sigjmp_buf env, int retval);
```

从不返回。

`longjmp` 函数从 `env` 缓冲区中恢复调用环境, 然后触发一个从最近一次初始化 `env` 的 `setjmp` 调用的返回。然后 `setjmp` 返回, 并带有非零的返回值 `retval`。

第一眼看过去, `setjmp` 和 `longjmp` 之间的相互关系令人迷惑。`setjmp` 函数只被调用一次, 但返回多次: 一次是当第一次调用 `setjmp`, 而调用环境保存在缓冲区 `env` 中时, 一次是为每个相应的 `longjmp` 调用。另一方面, `longjmp` 函数被调用一次, 但从不返回。

非本地跳转的一个重要应用就是允许从一个深层嵌套的函数调用中立即返回, 通常是由检测到某个错误情况引起的。如果在一个深层嵌套的函数调用中发现了一个错误情况, 我们可以使用非本地跳转直接返回到一个普通的本地化的错误处理程序, 而不是费力地解开调用栈。

图 8-43 展示了一个示例, 说明这可能是如何工作的。`main` 函数首先调用 `setjmp` 以保存当前的调用环境, 然后调用函数 `foo`, `foo` 依次调用函数 `bar`。如果 `foo` 或者 `bar` 遇到一个错误, 它们立即通过一次 `longjmp` 调用从 `setjmp` 返回。`setjmp` 的非零返回值指明了错误类型, 随后可以被解码, 且在代码中的某个位置进行处理。

code/ecf/setjmp.c

```
1  #include "csapp.h"
2
3  jmp_buf buf;
4
5  int error1 = 0;
6  int error2 = 1;
7
8  void foo(void), bar(void);
9
```

图 8-43 非本地跳转的示例。本示例表明了使用非本地跳转来从深层嵌套的函数调用中的错误情况恢复, 而不需要解开整个栈的基本框架


```
10  int main()
11  {
12      switch(setjmp(buf)) {
13          case 0:
14              foo();
15              break;
16          case 1:
17              printf("Detected an error1 condition in foo\n");
18              break;
19          case 2:
20              printf("Detected an error2 condition in foo\n");
21              break;
22          default:
23              printf("Unknown error condition in foo\n");
24      }
25      exit(0);
26  }
27
28  /* Deeply nested function foo */
29  void foo(void)
30  {
31      if (error1)
32          longjmp(buf, 1);
33      bar();
34  }
35
36  void bar(void)
37  {
38      if (error2)
39          longjmp(buf, 2);
40  }
```

code/ecf/setjmp.c

图 8-43 (续)

`longjmp` 允许它跳过所有中间调用的特性可能产生意外的后果。例如，如果中间函数调用中分配了某些数据结构，本来预期在函数结尾处释放它们，那么这些释放代码会被跳过，因而会产生内存泄漏。

非本地跳转的另一个重要应用是使一个信号处理程序分支到一个特殊的代码位置，而不是返回到被信号到达中断了的指令的位置。图 8-44 展示了一个简单的程序，说明了这种基本技术。当用户在键盘上键入 `Ctrl+C` 时，这个程序用信号和非本地跳转来实现软重启。`sigsetjmp` 和 `siglongjmp` 函数是 `setjmp` 和 `longjmp` 的可以被信号处理程序使用的版本。

在程序第一次启动时，对 `sigsetjmp` 函数的初始调用保存调用环境和信号的上下文（包括待处理的和被阻塞的信号向量）。随后，主函数进入一个无限处理循环。当用户键入 `Ctrl+C` 时，内核发送一个 `SIGINT` 信号给这个进程，该进程捕获这个信号。不是从信号处理程序返回，如果是这样那么信号处理程序会将控制返回给被中断的处理循环，反之，处理程序完成一个非本地跳转，回到 `main` 函数的开始处。当我们在系统上运行这个程序时，得到以下输出：

```

linux> ./restart
starting
processing...
processing...
Ctrl+C
restarting
processing...
Ctrl+C
restarting
processing...

```

关于这个程序有两件很有趣的事情。首先，为了避免竞争，必须在调用了 `sigsetjmp` 之后再设置处理程序。否则，就会冒在初始调用 `sigsetjmp` 为 `siglongjmp` 设置调用环境之前运行处理程序的风险。其次，你可能已经注意到了，`sigsetjmp` 和 `siglongjmp` 函数不在图 8-33 中异步信号安全的函数之列。原因是一般来说 `siglongjmp` 可以跳到任意代码，所以我们必须小心，只在 `siglongjmp` 可达的代码中调用安全的函数。在本例中，我们调用安全的 `sio_puts` 和 `sleep` 函数。不安全的 `exit` 函数是不可达的。

```

1  #include "csapp.h"
2
3  sigjmp_buf buf;
4
5  void handler(int sig)
6  {
7      siglongjmp(buf, 1);
8  }
9
10 int main()
11 {
12     if (!sigsetjmp(buf, 1)) {
13         Signal(SIGINT, handler);
14         Sio_puts("starting\n");
15     }
16     else
17         Sio_puts("restarting\n");
18
19     while(1) {
20         Sleep(1);
21         Sio_puts("processing...\n");
22     }
23     exit(0); /* Control never reaches here */
24 }

```

code/ecf/restart.c

图 8-44 当用户键入 Ctrl+C 时，使用非本地跳转来重启动它自身的程序

旁注 C++ 和 Java 中的软件异常

C++ 和 Java 提供的异常机制是较高层次的，是 C 语言的 `setjmp` 和 `longjmp` 函数的更加结构化的版本。你可以把 `try` 语句中的 `catch` 子句看做类似于 `setjmp` 函数。相似地，`throw` 语句就类似于 `longjmp` 函数。

8.7 操作进程的工具

Linux 系统提供了大量的监控和操作进程的有用工具。

STRACE：打印一个正在运行的程序和它的子进程调用的每个系统调用的轨迹。对于好奇的学生而言，这是一个令人着迷的工具。用-static 编译你的程序，能得到一个更干净的、不带有大量与共享库相关的输出的轨迹。

PS：列出当前系统中的进程(包括僵死进程)。

TOP：打印出关于当前进程资源使用的信息。

PMAP：显示进程的内存映射。

/proc：一个虚拟文件系统，以 ASCII 文本格式输出大量内核数据结构的内容，用户程序可以读取这些内容。比如，输入“cat/proc/loadavg”，可以看到你的 Linux 系统上当前的平均负载。

8.8 小结

异常控制流(ECF)发生在计算机系统的各个层次，是计算机系统中提供并发的基本机制。

在硬件层，异常是由处理器中的事件触发的控制流中的突变。控制流传递给一个软件处理程序，该处理程序进行一些处理，然后返回控制给被中断的控制流。

有四种不同类型的异常：中断、故障、终止和陷阱。当一个外部 I/O 设备(例如定时器芯片或者磁盘控制器)设置了处理器芯片上的中断管脚时，(对于任意指令)中断会异步地发生。控制返回到故障指令后面的那条指令。一条指令的执行可能导致故障和终止同步发生。故障处理程序会重新启动故障指令，而终止处理程序从不将控制返回给被中断的流。最后，陷阱就像是用来实现向应用提供到操作系统代码的受控的入口点的系统调用的函数调用。

在操作系统层，内核用 ECF 提供进程的基本概念。进程提供给应用两个重要的抽象：1)逻辑控制流，它提供给每个程序一个假象，好像它是在独占地使用处理器，2)私有地址空间，它提供给每个程序一个假象，好像它是在独占地使用主存。

在操作系统和应用程序之间的接口处，应用程序可以创建子进程，等待它们的子进程停止或者终止，运行新的程序，以及捕获来自其他进程的信号。信号处理的语义是微妙的，并且随系统不同而不同。然而，在与 Posix 兼容的系统上存在着一些机制，允许程序清楚地指定期望的信号处理语义。

最后，在应用层，C 程序可以使用非本地跳转来规避正常的调用/返回栈规则，并且直接从一个函数分支到另一个函数。

参考文献说明

Kerrisk 是 Linux 环境编程的完全参考手册[62]。Intel ISA 规范包含对 Intel 处理器上的异常和中断的详细讨论[50]。操作系统教科书[102, 106, 113]包括关于异常、进程和信号的其他信息。W. Richard Stevens 的[111]是一本有价值的和可读性很高的经典著作，是关于如何在应用程序中处理进程和信号的。Bovet 和 Cesati[11]给出了一个关于 Linux 内核的非常清晰的描述，包括进程和信号实现的细节。

家庭作业

• 8.9 考虑四个具有如下开始和结束时间的进程：

进程	开始时间	结束时间
A	5	7
B	2	4
C	3	6
D	1	8